

Storage Wars

MariaDB 13.0 stops overwriting its history, and Cloudflare's warehouse has no meter running

Issue #2

Tue, 06 Oct 2026

~37 min read

Level 3/5

Today's two stories

1. **MariaDB 13.0 is stable, and its most important feature is one you won't see in a query.** A new `innodb_log_archive` switch lets InnoDB keep its write-ahead log forever instead of overwriting it, which is the foundation for fast point-in-time recovery. You also get `UPDATE ... RETURNING`, but there are real catches, starting with the fact that 13.0 is a rolling release that gets no bug-fix updates.
2. **Cloudflare Basin is generally available.** Ingest, an Apache Iceberg catalog and a serverless SQL engine, all sitting on R2 storage with no egress fees and no hourly warehouse bill. We look at how it works under the hood, what it costs, and who it's actually for.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 MariaDB 13.0: the database that stopped overwriting its own history

MARIADB COMMUNITY SERVER 13.0.2 · GA ON 15 SEP 2026 · ROLLING RELEASE · ~16 MIN READ

Every InnoDB database you have ever run keeps a diary of every change it makes, and then, soon afterwards, tears the pages out and writes over them. That's by design: the diary only exists to survive a crash, and a fixed-size notebook is fast. But it means that if someone runs `DELETE FROM orders` without a `WHERE` at 14:03, the database itself has already forgotten how things looked at 14:02. MariaDB 13.0 adds a switch that tells InnoDB to stop tearing pages out. It's a small variable with a big idea behind it, and it ships alongside a long-requested SQL feature and a release model you need to understand before you put 13.0 anywhere near production.

WHAT HAPPENED

MariaDB Community Server **13.0.2** was released on **15 September 2026** as the first stable (GA) build of the 13.0 series.¹ MariaDB plc announced it the same day,² and the MariaDB Foundation followed on 17 September with "MariaDB 13.0 Is Now Stable", noting that the series "started its journey in March 2026 with the Preview".³

The official "changes and improvements" page lists the headline items:⁴

- **InnoDB log archiving** with a new `innodb_log_archive` variable, which "enables point-in-time recovery and incremental backups".

- **UPDATE ... RETURNING** for single-table updates, plus a new `OLD_VALUE()` function so you can read both the before and after values in one statement.
- **Oracle PL/SQL compatibility** work: `TYPE ... IS REF CURSOR` and `RECORD` types in routine parameters and function return types, so far only inside packages.
- **Smaller quality-of-life items:** `INFORMATION_SCHEMA.SYSTEM_VARIABLES` now says whether a variable is deprecated and what replaces it; optimizer trace shows which statistics it used; `QB_NAME()` hints can reach into views and common table expressions (`WITH` clauses); Performance Schema digests switch from MD5 to the `XXH3_128` hash; and the default `binlog_row_event_max_size` rises to 64k.

The release announcement is blunt about one thing that matters more than any feature: "MariaDB Community Server 13.0 GA is a rolling release and will not receive further bug-fix releases."²

We'll spend most of this story on log archiving, because it changes something fundamental about how InnoDB stores its history. But first, the release model, because it decides whether you should be running 13.0 at all.

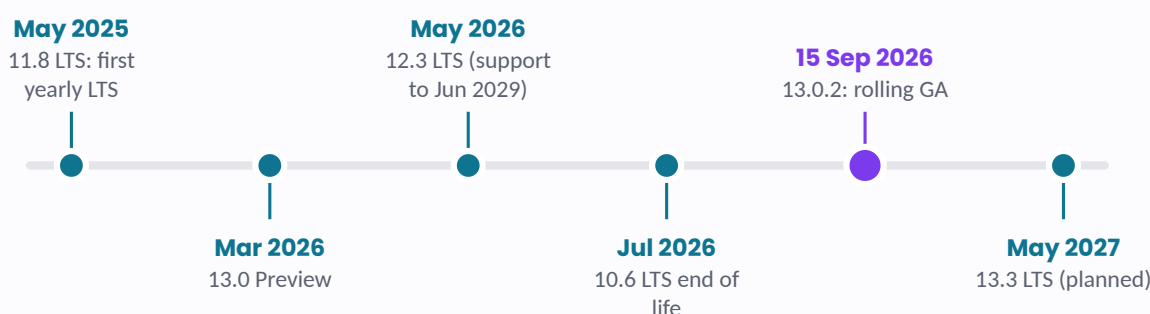
THE BACKGROUND YOU NEED

Rolling releases and LTS: which MariaDB are you supposed to run?

In February 2025 MariaDB plc announced that MariaDB Community Server would get a new **long-term support (LTS) release** *≈ a version that receives bug and security fixes for several years, meant for production* "every May", with quarterly **rolling releases** *≈ short-lived versions that ship new features quickly but are replaced by the next one instead of being patched* in between.⁵ The numbering follows a pattern: the LTS is "the last in line with the same major version number".⁵ So 12.0, 12.1 and 12.2 were rolling releases and **12.3 was the LTS** (May 2026), and 13.0, 13.1 and 13.2 will be rolling releases on the way to **13.3 LTS**, planned for May 2027.^{5,2}

MariaDB 12.3's first stable build, 12.3.2, arrived in late May 2026 with support until **June 2029**.⁶ At the other end of the scale, the long-lived 10.6 LTS reached the end of community maintenance on 6 July 2026, with a final 10.6.28 build on 13 August.⁷

Where 13.0 sits in MariaDB's release train



From MariaDB's yearly LTS announcement, the 13.0 GA post, Linuxiac on 12.3 and the 10.6 EOL report.

What does "rolling" mean in practice? If you find a bug in 13.0, the fix arrives in 13.1, together with whatever new features 13.1 brings. There is no 13.0.3 to apply quietly on a Tuesday. That's fine for development, for

trying features early, and for teams that upgrade every quarter anyway. For a production database you want to leave alone, the answer is still the LTS: **12.3 today, 13.3 next year**. Everything in this story is something you can test now and plan for in 13.3.

How InnoDB keeps your data safe: the redo log

To understand log archiving, you need to know what InnoDB does on every commit.

InnoDB doesn't write your changed rows straight to the table files. That would mean random writes all over the disk on every commit, which is slow. Instead, it changes the pages in memory, in the **buffer pool** *≈ InnoDB's in-memory cache of table and index pages*, and appends a compact description of the change to the **redo log** *≈ InnoDB's write-ahead log: a sequential record of every change, written before the changed pages reach the data files*. Writing sequentially to one file is fast, and once the log record is safely on disk, the commit is durable. The changed in-memory pages, called **dirty pages** *≈ pages in the buffer pool that have been modified but not yet written back to the data files*, get written to the table files later, in the background.

If the server crashes, the data files are out of date, but the redo log isn't. On restart, InnoDB replays the log and brings every page back to its committed state. This pattern is called **write-ahead logging (WAL)** *≈ the rule that a change must be recorded in a log before the data file itself is changed, so a crash can always be repaired by replaying the log*, and almost every serious database uses it.

Every log record gets a **log sequence number (LSN)** *≈ an ever-increasing number that marks a position in the redo log; a bigger LSN means a later change*. Periodically InnoDB takes a **checkpoint** *≈ a marker in the log saying "every change before this LSN is already in the data files", so recovery can start from here*. MariaDB's documentation says checkpoints are scheduled roughly "7 seconds apart when the server is very active".⁸

Here's the key detail. In MariaDB the redo log is a single file, `ib_logfile0`, and it's **circular** *≈ a fixed-size file where writing wraps around to the beginning and overwrites the oldest records*: "older entries get overwritten by newer ones".⁸ Anything before the last checkpoint is no longer needed for crash recovery, so it's fair game. Its size is `innodb_log_file_size`, which you can change on a running server since MariaDB 10.9.⁸ When the log is close to full, InnoDB starts "forcing checkpoints and flushing dirty pages" so it can reuse the space.⁸

That's a brilliant design for crash recovery and a terrible one for history. The log contains, physically, every change the database made, but only for the recent past.

How point-in-time recovery works today

Point-in-time recovery (PITR) *≈ restoring a database to its exact state at a chosen moment, for example one second before a bad DELETE* needs two things: a full backup, and a continuous record of every change made after it.

On MariaDB and MySQL, that record has traditionally been the **binary log (binlog)** *≈ the server's log of committed changes, used for replication and recovery; it records SQL statements or row changes, not physical page changes*. The documented procedure is: restore a `mariadb-backup` backup, read the binlog file and position the backup recorded in its `mariadb_backup_binlog_info` file, then replay the binlog up to your chosen moment.⁹

```
mysqlbinlog --start-position=321 \
  --stop-datetime="2019-06-28 12:00:00" \
  /var/lib/mysql/mariadb-node4.00001 > mariadb-binlog.sql
mariadb < mariadb-binlog.sql
```

That works, but it's **logical replay** *≈ re-executing recorded SQL statements or row events through the full SQL layer, as if a client sent them again*: every change goes back through parsing, locking and index maintenance, so a long stretch of binlog takes a long time to replay. It also only works if binary logging was switched on in the first place.

PostgreSQL has a different answer. Its WAL is split into segment files, "normally 16MB apiece", which are normally recycled once they precede the last checkpoint.¹⁰ With **WAL archiving** *≈ copying each finished WAL file somewhere safe before the database would recycle it*, an `archive_command` copies every completed segment before it's recycled, and recovery means restoring a file-level backup and replaying the archived WAL "to bring the system to a current state" or to any earlier point.¹⁰ That's **physical replay** *≈ re-applying low-level page changes straight from the log, without going through the SQL layer*, which skips all of that SQL-level work.

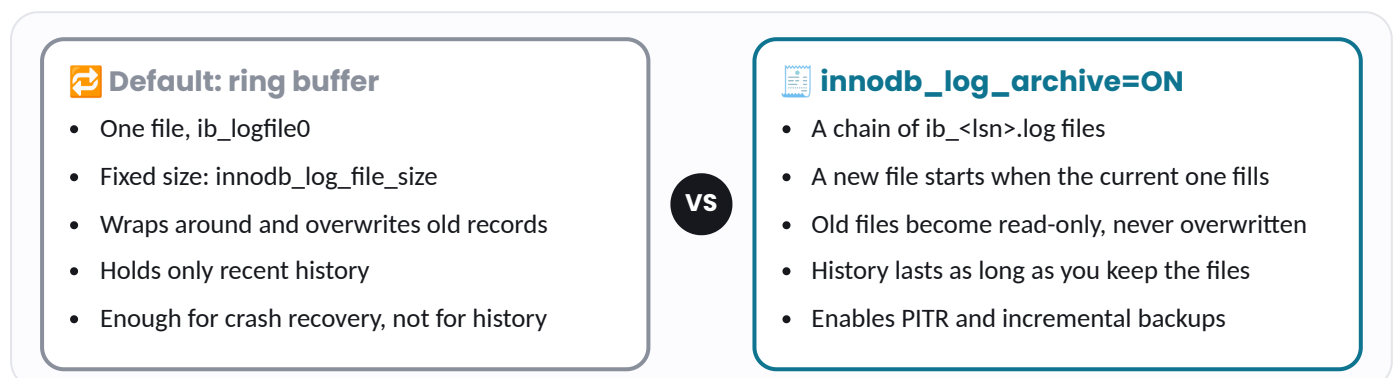
MySQL has a partial answer too. Since MySQL 8.0.17 it can archive redo log records to a directory set in `innodb_redo_log_archive_dirs`, started with `innodb_redo_log_archive_start()`. Its purpose is narrower: helping backup tools keep up when "the backup speed cannot keep up with the redo log generation", and the session that starts it "must remain open for the duration of the archiving".¹¹

MariaDB 13.0's log archiving is closer to the PostgreSQL model: the database itself keeps a permanent, physical change history.

HOW IT WORKS

From one ring to a chain of files

With `innodb_log_archive=ON`, InnoDB stops using the single `ib_logfile0` ring. As the 13.0 announcement puts it, "Instead of overwriting a fixed-size ring buffer (`ib_logfile0`), InnoDB writes a continuous, sequentially-named series of log files, retiring each one to read-only once a checkpoint completes in the next."²



The details, from the documentation and the design ticket (MDEV-37949):

- Naming.** Each file is called `ib_<lsn>.log`, where `<lsn>` is a 16-character hexadecimal number: the LSN at file offset `0x3000` (12,288 bytes), for example `ib_0000000000003000.log`.^{12,14} The file name tells you which part of history it covers, so tools don't need to open the file to know.

- **Header.** Each file has a 12 KiB header that records completed checkpoints as 64-bit offsets. With encryption off there's room for 1,536 checkpoint slots; after that, new checkpoints overwrite the last slot until the next file starts.¹² Recovery can therefore jump straight to a checkpoint instead of scanning the file.
- **Size.** Files are created at the configured `innodb_log_file_size`, with a 4 MB minimum.¹⁴ A size change takes effect when the current file fills and the next is created, and in archive mode there's no upper bound.¹²
- **Retirement.** When a checkpoint completes in a new file, the previous file becomes read-only and "can be archived to long-term storage".¹²
- **Switching it on.** The variable is dynamic, so you can enable it on a running server with `SET GLOBAL innodb_log_archive=ON;`. It defaults to `OFF`.^{12,14}

You can watch the archive with three status variables:¹²

```
SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM INFORMATION_SCHEMA.GLOBAL_STATUS
WHERE VARIABLE_NAME IN ( 'INNODB_LSN_ARCHIVED',
                        'INNODB_LSN_CURRENT',
                        'INNODB_LSN_LAST_CHECKPOINT' );
```

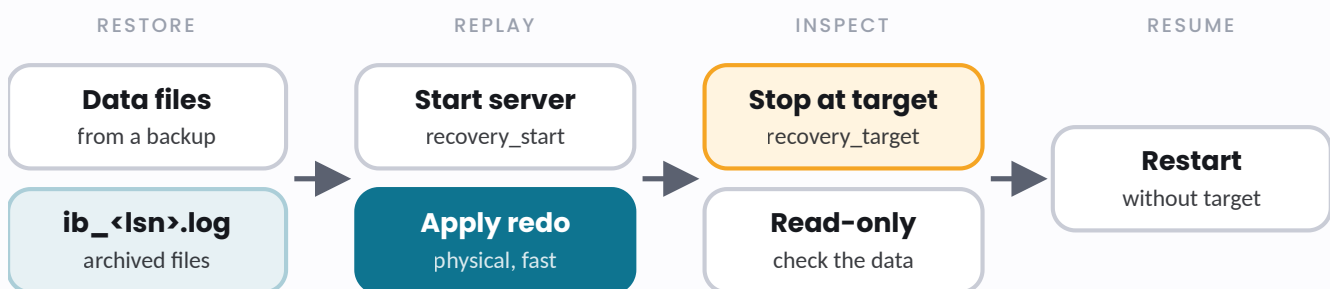
`Innodb_lsn_archived` is the LSN of the first checkpoint in the oldest archived file, the static lower bound of your history, and it "does not advance as new log records are written". `Innodb_lsn_current` is the latest log write, and `Innodb_lsn_last_checkpoint` is the most recent completed checkpoint. Those two keep moving.¹²

Recovering to a point in time

Recovery uses two startup variables:^{12,13}

- `innodb_log_recovery_start` : where replay begins. The default `0` means the latest completed checkpoint, "guaranteed to live in one of the last two `ib_lsn.log` files".
- `innodb_log_recovery_target` : where replay stops. The default `0` means "replay to the end". When it's non-zero, "persistent InnoDB tables become read-only and no log writes are allowed", so you can't accidentally carry on writing to a half-recovered database.

Point-in-time recovery with archived InnoDB logs



Procedure from MariaDB's PITR with InnoDB log archiving docs. No shipped backup tool automates this yet.

The documented procedure: put the restored InnoDB data files and the matching `ib_<lsn>.log` files in the data directory, making sure the legacy `ib_logfile0` is **absent**; start the server with `innodb_log_archive=ON` and the two recovery variables on the command line or in the config file; check `Innodb_lsn_archived`, `Innodb_lsn_current` and the table contents; then restart without `innodb_log_recovery_target` to resume normal operation.¹³

How do you choose the target? This is the rough edge. The documentation's suggested sources for LSNs are the `Innodb_lsn_archived` status variable and the LSNs encoded in the archive file names.¹³ There's no "stop at 14:02:59" option like the binlog's `--stop-datetime`. Mapping a wall-clock moment to an LSN is left to you, or to future tooling.

Why this is the foundation for better backups

The design ticket makes the bigger goal clear: backup programs are meant to query `innodb_log_archive` and the archived-LSN status, enable archiving before a backup, and use the archive for both full and incremental backups.¹⁴ A physical backup tool copies data files while the server keeps running, so it must also capture the redo log written in the meantime, before the ring overwrites it. On a busy server, as Oracle's MySQL team put it, "the backup speed cannot keep up with the redo log generation", which is the race MySQL's 8.0.17 feature was built to solve.¹¹ With an archive, there's no race: the log a backup needs is sitting in read-only files. An [incremental backup](#) *≈ a backup that copies only what changed since the previous backup, instead of everything* could, in principle, be little more than "copy the new `ib_<lsn>.log` files".

That's the promise. The present is more modest, as the next section explains.

The other headline: UPDATE ... RETURNING

MariaDB already supported `RETURNING` on `INSERT` and `DELETE`.¹⁵ 13.0 adds it to single-table `UPDATE`, with a companion function, `OLD_VALUE()`, that returns a column's value from before the update.^{16,17}

Claim a job and see what changed, in one round trip

Before 13.0: two statements

```
START TRANSACTION;
SELECT id, attempts FROM jobs
  WHERE id = 42 FOR UPDATE;
UPDATE jobs
  SET attempts = attempts + 1,
      status = 'running'
  WHERE id = 42;
COMMIT;
```

13.0: UPDATE ... RETURNING

```
UPDATE jobs
  SET attempts = attempts + 1,
      status = 'running'
  WHERE id = 42
  RETURNING id,
      OLD_VALUE(status) AS was,
      status AS now,
      attempts;
```

Illustrative table. `RETURNING` and `OLD_VALUE()` syntax from MariaDB's `UPDATE` and `OLD_VALUE` docs.

The documentation's own example is simple: `UPDATE t SET a=a+1 RETURNING OLD_VALUE(a) AS old, a AS new;` returns 1 and 2.¹⁶ The point is fewer round trips and no read-then-write window: the row you get back is exactly the row the `UPDATE` produced. The limitation: "RETURNING works only for single tables", so a multi-table `UPDATE ... JOIN` can't use it yet.¹⁶

WHAT IT MEANS FOR YOU

If you run MariaDB in production, don't move to 13.0 for these features. It's a rolling release with no bug-fix updates,² and log archiving has no backup tool support yet (see below). Stay on 12.3 LTS or 11.8 LTS, and pencil in **13.3 LTS** (planned for May 2027) as the version where log archiving and `UPDATE ... RETURNING` will be available with long-term support.⁵ If you're still on 10.6, that's the real emergency: community maintenance ended on 6 July 2026.⁷

Watch out for

- **mariadb-backup breaks with archiving on.** The docs say `mariadb-backup` "only supports" the legacy `ib_logfile0` format and "fails when the server is running with `innodb_log_archive=ON`", and that no shipped backup tool generates or restores backups in the new format yet.^{12,13} Turning it on today can break your nightly backup job.
- **The server never deletes archived files.** Disk usage grows until you move or delete them, and before deleting you must check that `Innodb_lsn_last_checkpoint` isn't older than the start LSN of the first writable `ib_<lsn>.log` file.¹² Monitor it like a PostgreSQL WAL archive.
- **DDL isn't fully covered.** Archiving tracks InnoDB changes only, not `.frm` files or non-InnoDB metadata, so point-in-time recovery of DDL is "limited".^{12,13}
- **Encryption is frozen while archiving.** You can't change `innodb_encrypt_log` while archiving is on; changing it means disabling archiving and restarting, which "permanently" discards the log history.¹²
- **No time-based targets.** You recover to an LSN, not a timestamp.¹³
- **The DuckDB engine is not in 13.0 packages.** Some write-ups describe 13.0 as bringing DuckDB integration in "gamma" status.¹⁸ MariaDB's own documentation for the DuckDB storage engine says it is "experimental at alpha maturity", "not yet merged into MariaDB Server `main`", not built into MariaDB releases, and not signed in any official package". You'd install it from CI builds or build it yourself, and `plugin-maturity=alpha` is required to load it.¹⁹
- **Atomic `CREATE OR REPLACE TABLE` : check before relying on it.** It's listed in the 13.0 changes page (MDEV-25292),⁴ but when we opened that ticket it showed status "Stalled" with a fix version of "13.2 Rolling Release".²⁰ Test the crash-safety behaviour yourself before you count on it.

If you build backup or recovery tooling, this is the feature to start prototyping against now. The file format, naming and status variables are documented,¹² and the design ticket explicitly expects backup programs to build on them.¹⁴

If you're an application developer, `UPDATE ... RETURNING` is the change you'll actually touch. Job queues ("claim the next job and tell me its attempt count"), counters ("increment and return the new value"), and

optimistic state machines ("move from `pending` to `paid` and show me the old status") all get simpler. Just remember it's 13.0+ only, so code that must also run on 12.3 LTS can't rely on it.

If you're hoping to run transactional and analytical queries in one database, you may have read that 13.0 brings a DuckDB engine. As the "Watch out" box above explains, it isn't in the 13.0 packages yet.

WHAT HAPPENS NEXT

Under the yearly model, more rolling releases (13.1, 13.2) will come before **13.3 LTS**, planned for May 2027.⁵ The 13.0 announcement calls these "quarterly updates leading toward our next Long-Term Support (LTS) version (13.3)".² The things to watch along the way: `mariadb-backup` support for the archived format, some way to recover to a timestamp instead of an LSN, `RETURNING` for multi-table updates, and whether the DuckDB engine graduates from alpha into the main tree.

Words from this story

Long-term support (LTS) release: a version that gets bug and security fixes for years; the one to run in production.

Rolling release: a short-lived version that ships features quickly and is replaced by the next one instead of being patched.

Buffer pool: InnoDB's in-memory cache of table and index pages.

Redo log: InnoDB's write-ahead log, a sequential record of every change written before the pages reach the data files.

Dirty pages: pages changed in memory but not yet written back to the data files.

Write-ahead logging (WAL): log the change first, change the data file later, so a crash can be repaired by replaying the log.

Log sequence number (LSN): an ever-increasing position in the redo log.

Checkpoint: a log marker saying every change before it is already in the data files.

Circular log: a fixed-size file that wraps around and overwrites its oldest records.

Point-in-time recovery (PITR): restoring a database to its exact state at a chosen moment.

Binary log (binlog): the server's log of committed changes, used for replication and recovery.

Logical replay: re-executing recorded statements or row events through the SQL layer.

WAL archiving: copying finished log files somewhere safe before the database reuses them.

Physical replay: re-applying low-level page changes straight from the log.

Incremental backup: a backup that copies only what changed since the previous one.

STORY 2 Cloudflare Basin: a data warehouse with no meter running

CLOUDFLARE BASIN · GENERALLY AVAILABLE 1 OCT 2026 · APACHE ICEBERG ON R2 · ~14 MIN READ

Most analytics stacks charge you twice before you've asked a single question. You pay a warehouse by the hour to sit there waiting for queries, and you pay a cloud provider by the gigabyte every time data leaves its network. Cloudflare's pitch with Basin, which went GA last week, is to remove both: no cluster to keep warm, and no fee to read your data from anywhere else. Under the marketing is a genuinely interesting piece of engineering, a query engine that plans against open-format metadata and runs across Cloudflare's network. And under that is the question every data team should ask: what am I actually paying for, and what can't it do yet?

WHAT HAPPENED

On **1 October 2026**, Cloudflare announced the general availability of **Cloudflare Basin**, "a serverless data analytics platform built on Apache Iceberg and R2 Object Storage". During its beta it was called the Cloudflare Data Platform.²¹ Basin is three products that previously had their own names:^{21,22}

- **Basin Pipelines** (formerly Cloudflare Pipelines) ingests events from **Workers** *≈ Cloudflare's serverless functions, which run your JavaScript or TypeScript on Cloudflare's network*, HTTP endpoints or Cloudflare Logpush, transforms them with SQL, and writes them out as Iceberg tables or as JSON or Parquet files in R2.
- **Basin Catalog** (formerly R2 Data Catalog) manages Iceberg table metadata and maintains the tables automatically.
- **Basin SQL** (formerly R2 SQL) is a "serverless, distributed SQL query engine" for those Iceberg tables.

Existing setups under the old names keep working.²¹ The same day, Cloudflare raised the ingest limit for each Pipelines stream to **1 GB/s**, up from 5 MB/s.²² Billing for all three components had already been switched on in August.²²

Cloudflare's CTO, Dane Knecht, summed up the pitch: "Developers shouldn't need to become data infrastructure operators just to answer questions from their own data."²³

THE BACKGROUND YOU NEED

Three ideas that make this possible

Object storage without egress fees. **Object storage** *≈ cheap, effectively unlimited storage for whole files ("objects") accessed over HTTP, like Amazon S3* is where most modern analytics data lives. Most clouds charge an **egress fee** *≈ a per-gigabyte charge for moving data out of a cloud provider's network* when you read that data from outside their network. Amazon Athena's pricing page, for example, notes that you pay "standard S3 rates for storage, requests, and data transfer".²⁴ Cloudflare's R2 doesn't charge for egress at all. Its standard storage costs \$0.015 per GB-month, writes ("Class A" operations) cost \$4.50 per million and reads ("Class B") cost \$0.36 per million.²⁵ That one pricing decision is the foundation of everything in this story.

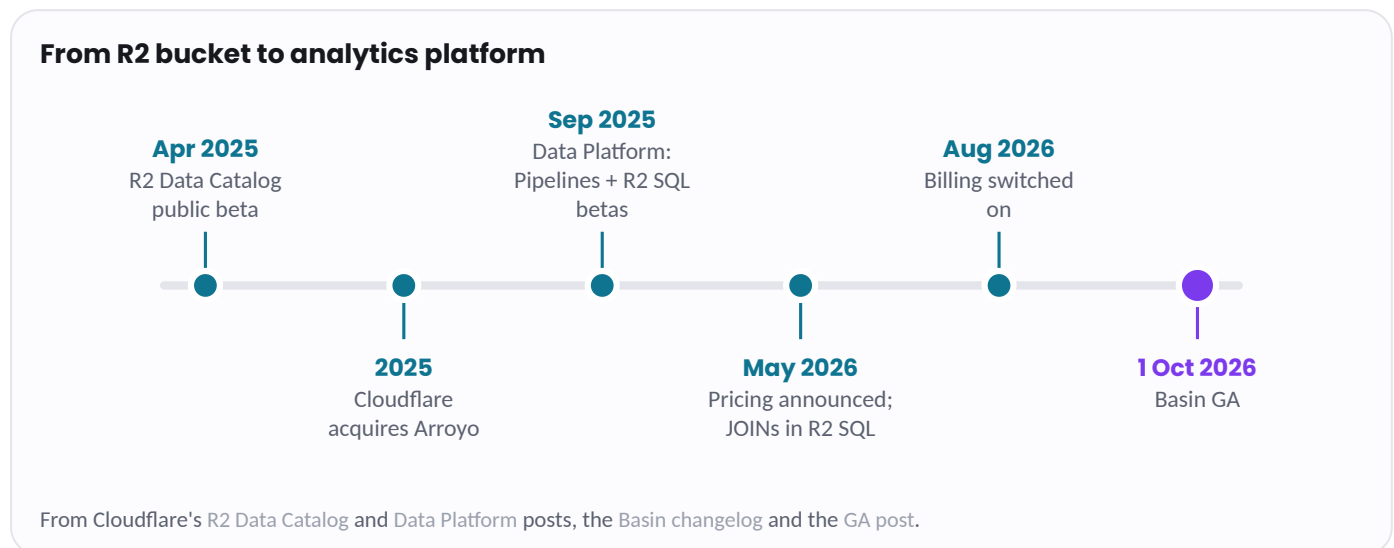
Columnar files. Analytics data is usually stored as **Parquet** *≈ an open, column-oriented file format: values of each column are stored together and compressed, so a query can read just the columns it needs* files. Each Parquet file is split into **row groups** *≈ chunks of rows inside a Parquet file; each records min/max statistics per column, so readers can skip chunks that can't match* .

Open table formats. A folder of Parquet files isn't a table. Who knows which files are current? What happens if two writers add files at once? **Apache Iceberg** *≈ an open table format that turns a collection of data files in object storage into a real table, with transactions, snapshots and schema changes* solves this with a layer of metadata files. Cloudflare's own description: Iceberg brings "database-like functionality" to Parquet files, "including ACID transactions, time travel capabilities, and schema evolution", and it's read by engines such as Spark, Trino, Snowflake, DuckDB and ClickHouse.²⁶

Iceberg tracks a table as a series of **snapshots** *≈ immutable versions of a table; each write creates a new one, so you can query or roll back to an older version* . Each snapshot points to a **manifest list**, which points to **manifests**, which list the actual data files along with statistics about them. Something has to know which metadata file is the current one, and that's the job of a **catalog** *≈ the service that records where each table's current metadata lives, so every engine sees the same version* . Without one, as Cloudflare put it, engines "might access outdated versions" or write new data where nobody can find it.²⁶ The **Iceberg REST catalog** *≈ a standard HTTP API for catalogs, so any compliant engine can find and update tables without vendor-specific code* is the standard way engines talk to a catalog, and it's what Basin Catalog exposes.²⁷

How Cloudflare got here

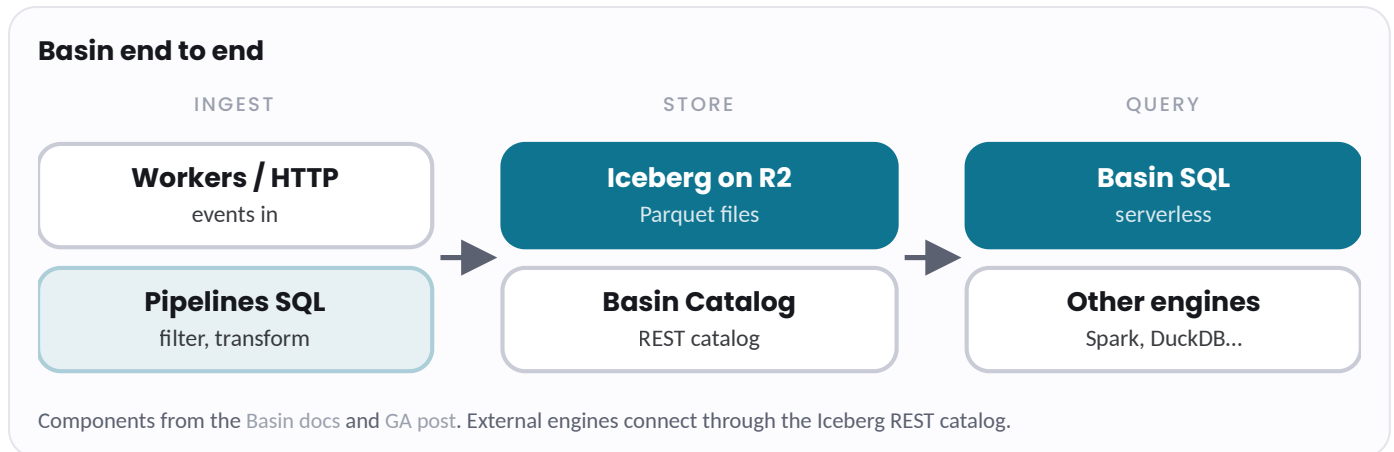
Basin wasn't built in one go. It's the result of roughly 18 months of separate launches that have now been bundled together:



The R2 Data Catalog beta launched on 10 April 2025 with the line "as always with R2, there are no egress fees".²⁶ On 25 September 2025 Cloudflare announced the "Cloudflare Data Platform": Pipelines and R2 SQL in open beta, and automatic compaction for the catalog. Pipelines was "built on top of Arroyo, a stream processing engine we acquired earlier this year".²⁸ Through 2026 the changelog filled in the gaps: nearly 200 new SQL functions in March, **JOIN** s, subqueries and CTEs in May, window functions and set operations in

June, automatic cleanup of unreferenced data files in April, read-only catalog tokens in July, and pricing announced in May with billing from 3 August.²²

HOW IT WORKS



1. Pipelines: getting events in

Pipelines has three building blocks.²⁹ **Streams** are "durable, buffered queues that receive events via HTTP endpoints or Worker bindings". **Pipelines** connect streams to sinks "with SQL transformations that validate, filter, transform, and enrich your data at ingestion time". **Sinks** write the results as Iceberg tables in Basin Catalog or as Parquet or JSON files, and the docs promise "exactly-once delivery to R2".²⁹ **Exactly-once delivery** *≈ a guarantee that each event ends up in the destination one time, never lost and never duplicated, even if parts of the system retry* matters here, because duplicated events in an analytics table quietly inflate every count.

From a Worker, sending an event is one line, because a **binding** *≈ a Workers configuration entry that connects your code to another Cloudflare resource without you handling credentials* handles authentication. This is the example from Cloudflare's getting-started guide:³⁰

```
export default {
  async fetch(request, env, ctx) {
    await env.EVENTS.send([ { path: new URL(request.url).pathname } ]);
    return new Response("Hello World!");
  },
};
```

The sink doesn't write a file per event. It batches them and writes on a **roll interval** *≈ how long a sink collects events before writing them out as a file*. The guide sets it to 60 seconds and warns you to "wait a few minutes for the first file to be written" before querying.³⁰ So Basin is **near-real-time**: think a minute or more of delay, not milliseconds.

Limits per account are 20 streams, 20 pipelines and 20 sinks, with a 5 MB maximum payload per ingestion request and 1 GB/s per stream.³¹

2. Catalog: keeping tables healthy

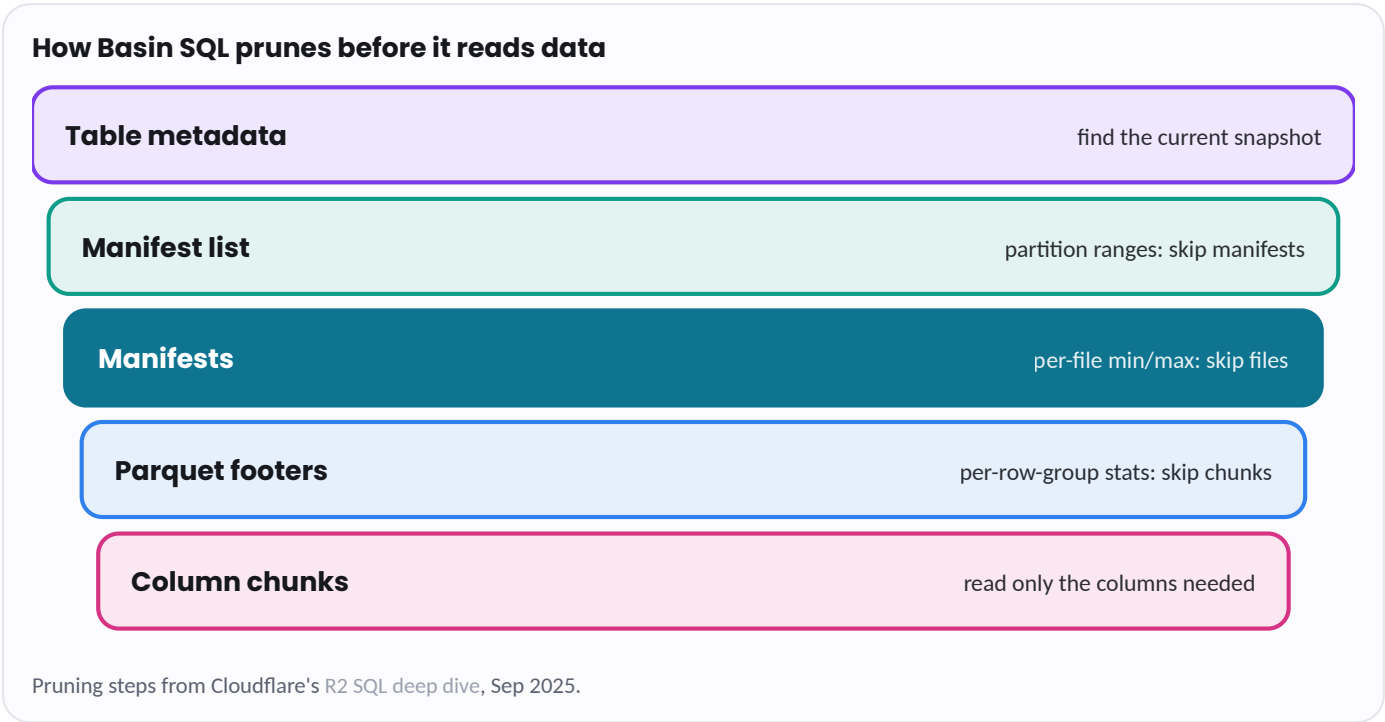
Streaming ingestion creates a classic problem. Every roll interval writes new, small files, and Iceberg adds a snapshot for every commit. At a 60-second roll interval that's up to 1,440 commits a day per sink, by our arithmetic, each adding small files and another snapshot to the metadata history. Cloudflare's docs list the consequences of this [small files problem](#) *≈ the slowdown caused by having many tiny data files: more files to open, more metadata to plan with, and worse compression* : slower queries, more planning overhead and worse compression.³²

Basin Catalog fixes this with automatic table maintenance:^{21,32}

- [Compaction](#) *≈ rewriting many small data files into fewer large ones, without changing the table's contents* , with a target file size between 64 MB and 512 MB. The docs suggest 64–128 MB for latency-sensitive tables, 128–256 MB for streaming ingest and 256–512 MB for heavy analytical queries.
- **Snapshot expiration**, which by default removes snapshots older than 30 days while always keeping at least the last 5.
- **Unreferenced data-file cleanup**, which deletes files no snapshot points to any more (added in April 2026).²²
- **Manifest optimization**, which reduces "metadata I/O" during planning (added in July 2026).²²

3. Basin SQL: how a serverless engine answers a query

This is the most interesting part technically. Cloudflare's engineers explained the design when it launched as R2 SQL in September 2025.³³ The trick is to do as little I/O as possible by reading metadata first, and Iceberg's metadata tree has statistics at every level:



The planner reads the table's current snapshot, uses partition statistics in the manifest list to throw away whole manifests, uses per-file column statistics in the manifests to throw away individual Parquet files, and then uses Parquet footer statistics to skip row groups.³³ Cloudflare's example: if a query asks for `http_status`

= 500 and a file's statistics say that column ranges from 200 to 404, "that entire file can be skipped".³³ This is called **pruning** *≈ skipping data that can't match a query by looking only at its statistics*, and it's why table layout and compaction matter so much: well-sorted, well-sized files have tight min/max ranges that prune well.

Planning and execution overlap. Rather than finishing the plan first, "the planner's job is to produce a stream of work units for the executor to consume as soon as they are available."³³ For `ORDER BY ... LIMIT` queries, it reads the most promising manifests first and stops early once nothing left could beat the results it already has, though at launch this worked only for columns in the table's partition key.³³

Execution is distributed. A coordinator on Cloudflare's network hands work to query workers, which run **Apache DataFusion** *≈ an open-source query engine written in Rust that executes SQL on columnar data* over Parquet row groups and send results back in **Apache Arrow** *≈ a standard in-memory columnar data format, used to move tables between systems without converting them* format over gRPC.³³ Because Parquet is columnar, "if the query only ever uses five columns, we can only read them and skip reading the remaining fifty."³³

At GA, Cloudflare lists 190+ functions plus joins (inner, outer, semi, anti), subqueries, window functions, CTEs, set operations, `QUALIFY`, grouping sets, rollups and cubes.²¹ You can run queries from the dashboard or from

Wrangler *≈ Cloudflare's command-line tool for managing Workers and related products* :³⁰

```
npx wrangler basin sql query "<WAREHOUSE_NAME>" \  
"SELECT path, COUNT(*) AS requests  
FROM default.request_events  
GROUP BY path ORDER BY requests DESC"
```

What it costs

Basin is billed only when it "ingests, processes, or queries your data", with "no hourly charges".²¹ Here are the list prices:^{34,35}

Basin list prices (beyond the monthly free allowance)

\$0.04/GB

Pipelines **SQL transforms**
(50 GB free)

\$0.06/GB

Sink to **Parquet/Iceberg**
(\$0.03 JSON)

\$2.50/TB

Basin **SQL** compressed
data scanned (10 GB free)

\$9/M

Catalog **operations** (1M
free)

Compaction is billed too: \$0.005 per GB compacted (10 GB free) and \$2 per million objects (1 million free).³⁴

R2 storage and operations are billed separately on top.³⁴ Pipelines require the Workers Paid plan.²⁹

Cloudflare's own worked example: a pipeline that processes 500 GB a month and writes 300 GB to Iceberg costs **\$33.00**, made up of \$18.00 for transforms and \$15.00 for the sink.³⁶ For queries, a "GB scanned" means compressed bytes read from R2: a 100 MB Parquet file counts as 100 MB. Every query is billed for at least 10 MB, failed queries are free, and so are metadata commands like `EXPLAIN` and `DESCRIBE`.³⁵ By our arithmetic

from those list prices, a query that scans 40 GB of compressed Parquet costs about 10 cents, and a well-pruned query that touches 50 MB costs a fraction of a cent.

WHAT IT MEANS FOR YOU

Who it's for. Basin is aimed squarely at teams that already run on Cloudflare Workers and want analytics without hiring a data platform team: product events, request logs, billing metrics, telemetry. Cloudflare says early adopters created "tens of thousands of Pipelines" during the beta, and that its own billing and infrastructure teams use it for real-time e-commerce optimisation, long-term billing metrics and telemetry ingestion.²¹

In the real world

Anomaly, the company behind OpenCode, says it "moved our entire company's data pipeline to Basin Pipelines, Catalog, and SQL, replacing a complex AWS S3 and Athena setup with a cleaner, serverless architecture" (co-founder Dax Raad). **Bobsled**, which distributes data products to other platforms, uses Basin to make data "accessible in any region of every major data and AI platform" at "a fraction of the cost thanks to zero egress fees" (Julien Grobbelaar).²¹ Both are customer quotes in Cloudflare's own announcement, so treat them as testimonials, not benchmarks.

The S3 + Athena comparison is the natural one, and Anomaly's quote makes it explicitly. Both are serverless and both bill by data scanned. The differences are that R2 doesn't charge egress when other engines or clouds read the data,²⁵ that Basin bundles ingestion and table maintenance into the same product,²¹ and that the catalog is the open Iceberg REST standard.²⁷ Whether it's cheaper for you depends on your scan volumes and query patterns, so price your real workload with the published numbers rather than taking the headline at face value.

The lock-in question. Because Basin Catalog speaks the standard Iceberg REST protocol, the same tables are readable from other engines. Cloudflare publishes configuration examples for Spark, Snowflake and Pylceberg.²⁷ Combined with zero egress, that's the anti-lock-in argument: your data sits in an open format in a bucket you control, and you can point a different engine at it without paying to get it out.

When to think twice. If you need sub-second freshness, the roll-interval design means you'll wait a minute or more for new data.³⁰ If you need heavy write workloads from SQL, note that the pages we read describe Basin SQL as a query engine over tables that Pipelines and other Iceberg writers populate. And if you already have a Snowflake or Databricks estate, Basin is more likely to complement it, as cheap storage that those engines can read through the REST catalog,²⁷ than to replace it.

How to try it. Cloudflare's guide takes you from an empty project to a queryable Iceberg table with one Worker, one `wrangler basin pipelines setup` command and one SQL query.³⁰ With the 50 GB and 10 GB free allowances,³⁴ a small experiment will probably cost nothing beyond R2 storage.

Watch out for

- **Two different ingest numbers.** The GA post says "up to 3GB/s per stream",²¹ while the changelog and limits page say **1 GB/s**.^{22,31} Plan around 1 GB/s.
- **The price tables aren't the whole bill.** R2 storage (\$0.015/GB-month), writes and reads are charged on top,^{34,25} and every file a query opens is a Class B read operation.³⁵ Compaction is billed per GB and per object.³⁴
- **10 MB minimum per query.** Thousands of tiny dashboard queries add up faster than the per-TB price suggests.³⁵
- **Snapshot expiration limits time travel.** With the default 30-day / last-5 policy, you can't query a table as it was 6 weeks ago.³²
- **Some docs lag behind.** The table-maintenance page still lives under "R2 Data Catalog" and says orphaned files aren't cleaned up,³² which the April 2026 changelog entry changed.²² When docs disagree, trust the changelog.

WHAT HAPPENS NEXT

Cloudflare's stated roadmap includes support for the latest Iceberg spec, push-button ingestion sources and destinations, "adaptive" table-maintenance strategies, wider SQL compatibility, real-time processing with action triggers, and data compliance and sovereignty tools.²¹ The bigger question is whether zero-egress Iceberg storage becomes a neutral meeting point for other engines. If it does, the pressure lands on every cloud that still charges you to take your own data out.

Words from this story

Object storage: cheap, effectively unlimited file storage accessed over HTTP, like S3 or R2.

Egress fee: a per-gigabyte charge for moving data out of a cloud provider's network.

Parquet: an open, column-oriented, compressed file format for analytics.

Row groups: chunks of rows inside a Parquet file, each with per-column min/max statistics.

Apache Iceberg: an open table format that turns data files in object storage into a real table.

Snapshots: immutable versions of an Iceberg table, one per write.

Catalog: the service that records where each table's current metadata lives.

Iceberg REST catalog: the standard HTTP API engines use to find and update Iceberg tables.

Exactly-once delivery: every event lands once, never lost or duplicated.

Workers: Cloudflare's serverless functions.

Wrangler: Cloudflare's command-line tool.

Binding: a Workers config entry that connects code to a Cloudflare resource without handling credentials.

Roll interval: how long a sink collects events before writing a file.

Small files problem: the slowdown caused by many tiny data files.

Compaction: rewriting many small files into fewer large ones.

Pruning: skipping data that can't match a query, using only its statistics.

Apache DataFusion: an open-source Rust query engine for columnar data.

Apache Arrow: a standard in-memory columnar format for moving tables between systems.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **pgvector 0.8.7 (5 Oct)**: fixes a buffer overflow in IVFFlat index builds (CVE-2026-103484) that "can lead to arbitrary code execution", so upgrade the extension if you build IVFFlat indexes.³⁷
- **PostgreSQL 19**: no release candidate yet in what we checked: the roadmap still says "planned for October 2026" and points to Beta 4 (24 Sep), with the next minor releases due 12 November.³⁸
- **Meta's ZGateway (InfoQ, 28 Sep)**: a stateless proxy in front of the ZippyDB key-value store that handles over 1 billion operations per second and cuts persistent connections about 19x.³⁹
- **ClickHouse 26.9 (23 Sep)**: adds `LIMIT . . . AFTER/UNTIL` row-value boundaries, incremental refreshable materialized views, time-limited `CREATE TOKEN` credentials and disk spilling for `DISTINCT`.⁴⁰
- **Databricks (2 Oct)**: JDBC Unity Catalog connections are now generally available on serverless compute, modern runtimes and SQL warehouses.⁴¹
- **Confluent Current 2026 Americas**: set for 4–5 November in San Francisco, the first Current since IBM's acquisition of Confluent.⁴²

1. [MariaDB 13.0.2 release notes](#), MariaDB docs, 15 Sep 2026.
2. [Announcing MariaDB Community Server 13.0 GA](#), Ralf Gebhardt, MariaDB plc, 15 Sep 2026.
3. [MariaDB Foundation blog: posts tagged "mariadb"](#) ("MariaDB 13.0 Is Now Stable", Frédéric Descamps, 17 Sep 2026).
4. [MariaDB 13.0 Changes & Improvements](#), MariaDB docs.
5. [Announcing yearly LTS releases for MariaDB Community Server](#), MariaDB plc, 13 Feb 2025.
6. [MariaDB 12.3 LTS Debuts with Support Until June 2029](#), Linuxiac, 30 May 2026.
7. [MariaDB Q3 2026 maintenance releases and 10.6 end of life](#), LinuxCompatible, Aug 2026 (citing the MariaDB Foundation).
8. [InnoDB Redo Log](#), MariaDB docs.
9. [Point-In-Time Recovery \(PITR, mariadb-backup\)](#), MariaDB docs.
10. [Continuous Archiving and Point-in-Time Recovery \(PITR\)](#), PostgreSQL 19 documentation.
11. [MySQL InnoDB Redo Log Archiving](#), Oracle MySQL blog, 6 Jan 2023.
12. [InnoDB Log Archiving](#), MariaDB docs.
13. [Point-In-Time Recovery \(InnoDB Log Archiving\)](#), MariaDB docs.
14. [MDEV-37949: Implement innodb_log_archive](#), MariaDB Jira (fix version 13.0.1).
15. [INSERT...RETURNING](#), MariaDB docs.
16. [UPDATE](#), MariaDB docs.
17. [OLD_VALUE](#), MariaDB docs.
18. [MariaDB 13.0 Goes Stable with New SQL and InnoDB Features](#), Linuxiac, 18 Sep 2026.
19. [DuckDB Storage Engine](#), MariaDB docs.
20. [MDEV-25292: Atomic CREATE OR REPLACE TABLE](#), MariaDB Jira, viewed 6 Oct 2026.
21. [Introducing Cloudflare Basin: an open, serverless data platform, now generally available](#), Marc Selwan and Micah Wylde, Cloudflare, 1 Oct 2026.
22. [Basin changelog](#), Cloudflare Developers.
23. [Cloudflare Basin brings serverless data analysis to developers](#), Techzine, Oct 2026.
24. [Amazon Athena pricing](#), AWS.

25. [R2 pricing](#), Cloudflare Developers.
26. [R2 Data Catalog public beta](#), Cloudflare, 10 Apr 2025.
27. [Basin Catalog](#), Cloudflare Developers.
28. [Announcing the Cloudflare Data Platform](#), Cloudflare, 25 Sep 2025.
29. [Basin Pipelines](#), Cloudflare Developers.
30. [Basin getting started guide](#), Cloudflare Developers.
31. [Basin Pipelines limits](#), Cloudflare Developers.
32. [Table maintenance](#), Cloudflare Developers (R2 Data Catalog docs).
33. [R2 SQL: a deep dive into our new distributed query engine](#), Yevgen Safronov, Nikita Lapkov and Jérôme Schneider, Cloudflare, 25 Sep 2025.
34. [Basin pricing](#), Cloudflare Developers.
35. [Basin SQL pricing](#), Cloudflare Developers.
36. [Basin Pipelines pricing](#), Cloudflare Developers.
37. [pgvector 0.8.7 Released](#), postgresql.org, 5 Oct 2026.
38. [PostgreSQL Roadmap](#), postgresql.org.
39. [Meta's ZGateway Cuts ZippyDB Connections 19x While Handling 1B+ Operations per Second](#), InfoQ, 28 Sep 2026.
40. [ClickHouse Release 26.9](#), ClickHouse, 23 Sep 2026.
41. [Databricks release notes: October 2026](#), Databricks.
42. [What to expect from Confluent Current 2026 Americas](#), Computer Weekly, Sep 2026.